

Prog.1: WAP to input an integer array. Print all elements with their addresses.

Ans:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Elements with their addresses:\n");
    for (int i = 0; i < n; i++) {
        printf("Element %d is %d at address %p\n", i, arr[i],
(void*)&arr[i]);
    }

    return 0;
}
```

Prog.2: WAP to perform Linear Search.

Ans:

```
#include <stdio.h>

int main() {
    int n, search, found = 0;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &search);

    for (int i = 0; i < n; i++) {
        if (arr[i] == search) {
            printf("Element found at index %d\n", i);
            found = 1;
            break;
        }
    }
}
```

```
    if (!found) {
        printf("Element not found in the array.\n");
    }

    return 0;
}
```

Prog.3: WAP to find maximum and minimum number of an array.

Ans:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int max = arr[0], min = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    printf("Maximum element: %d\n", max);
    printf("Minimum element: %d\n", min);

    return 0;
}
```

Prog.4: WAP to input and output 3x3 matrix. Print it and its Transpose.

Ans:

```
#include <stdio.h>

int main() {

    int matrix[3][3], transpose[3][3];
```

```

printf("Enter elements of the 3x3 matrix:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

printf("Original matrix:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}

// Transpose
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        transpose[j][i] = matrix[i][j];
    }
}

printf("Transpose of the matrix:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", transpose[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Prog.5: WAP to input and output 3x3 matrix. Print its both diagonal elements.

Ans:

```

#include <stdio.h>

int main() {
    int matrix[3][3];

    printf("Enter elements of the 3x3 matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    printf("Principal diagonal elements: ");
    for (int i = 0; i < 3; i++) {
        printf("%d ", matrix[i][i]);
    }
}

```

```

printf("\nSecondary diagonal elements: ");
for (int i = 0; i < 3; i++) {
    printf("%d ", matrix[i][2 - i]);
}

return 0;
}

```

Prog.6: WAP to input and output 5x5 matrix. Print its:

- a. Upper Triangular Matrix
- b. Lower Triangular Matrix

Ans:

```

#include <stdio.h>

int main() {
    int matrix[5][5];

    printf("Enter elements of the 5x5 matrix:\n");
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    printf("Upper Triangular Matrix:\n");
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (i <= j) {
                printf("%d ", matrix[i][j]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }

    printf("Lower Triangular Matrix:\n");
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (i >= j) {
                printf("%d ", matrix[i][j]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }

    return 0;
}

```

Prog.7: WAP to input & print its Tridiagonal Matrix.

Ans:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter the size of the matrix: ");
    scanf("%d", &n);

    int matrix[n][n];

    printf("Enter elements of the matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    printf("Tridiagonal Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j || i == j + 1 || i == j - 1) {
                printf("%d ", matrix[i][j]);
            } else {
                printf("0 ");
            }
        }
        printf("\n");
    }

    return 0;
}
```

Prog.8: WAP to input sparse matrix & print sparse & dense both matrices.

Ans:

```
#include <stdio.h>

int main() {
    int row, col, sparse[10][10], nonZeroCount = 0;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &row, &col);

    printf("Enter elements of the matrix:\n");
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
```

```

        scanf("%d", &sparse[i][j]);
        if (sparse[i][j] != 0) {
            nonZeroCount++;
        }
    }
}

printf("Dense Matrix:\n");
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        printf("%d ", sparse[i][j]);
    }
    printf("\n");
}

int sparseMatrix[nonZeroCount][3], k = 0;
printf("Sparse Matrix (Row, Column, Value):\n");
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        if (sparse[i][j] != 0) {
            sparseMatrix[k][0] = i;
            sparseMatrix[k][1] = j;
            sparseMatrix[k][2] = sparse[i][j];
            k++;
        }
    }
}

for (int i = 0; i < nonZeroCount; i++) {
    printf("%d %d %d\n", sparseMatrix[i][0], sparseMatrix[i][1],
sparseMatrix[i][2]);
}

return 0;
}

```

Prog.9: WAP to print the following menu with a single printf() function.

Ans:

```

#include <stdio.h>

int main() {
    printf("***M E N U**\n1. PUSH\n2. POP\n3. SHOW\n4. EXIT\nEnter your
choice(1...4):_");
    return 0;
}

```

Prog.10: Write a menu-driven program to show Stack Operations.

Ans:

```

#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push() {
    int val;
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        printf("Enter the value to push: ");
        scanf("%d", &val);
        top++;
        stack[top] = val;
        printf("%d pushed into the stack\n", val);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
    } else {
        printf("%d popped from the stack\n", stack[top]);
        top--;
    }
}

void show() {
    if (top == -1) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements are: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    int choice;
    while (1) {
        printf("\n**M E N U**\n1. PUSH\n2. POP\n3. SHOW\n4. EXIT\nEnter your
choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: show(); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Prog.11: Write a menu-driven program to show simple & circular queue operations.

Ans:

```
#include <stdio.h>
#define MAX 5

int queue[MAX], front = -1, rear = -1;

void enqueue() {
    int val;
    if ((rear + 1) % MAX == front) {
        printf("Queue is Full (Circular Queue Overflow)\n");
    } else {
        printf("Enter the value to enqueue: ");
        scanf("%d", &val);
        if (front == -1) front = 0;
        rear = (rear + 1) % MAX;
        queue[rear] = val;
        printf("%d enqueued to the circular queue\n", val);
    }
}

void dequeue() {
    if (front == -1) {
        printf("Queue is Empty (Circular Queue Underflow)\n");
    } else {
        printf("%d dequeued from the circular queue\n", queue[front]);
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % MAX;
        }
    }
}

void display() {
    if (front == -1) {
        printf("Queue is empty\n");
    } else {
        printf("Queue elements: ");
        int i = front;
        while (1) {
            printf("%d ", queue[i]);
            if (i == rear) break;
            i = (i + 1) % MAX;
        }
        printf("\n");
    }
}

int main() {
    int choice;
    while (1) {
        printf("\n**MENU**\n1. ENQUEUE\n2. DEQUEUE\n3. DISPLAY\n4. EXIT\nEnter your choice: ");
```



```

        scanf("%d", &choice);
        switch (choice) {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Prog.12: WAP to create and print Single Linked List without functions.

Ans:

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

int main() {
    int n, data;
    struct Node* head = NULL;
    struct Node* temp;
    struct Node* newNode;

    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter data for node %d: ", i + 1);
        scanf("%d", &newNode->data);
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            temp = head;
        } else {
            temp->next = newNode;
            temp = temp->next;
        }
    }
    printf("The linked list is: ");
    temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");

    return 0;}

```

